

项目申请书

项目名称	Jeandle Code Buffer 的精确分配机制
项目编号	26c280002
申请人	黄治文
学校	上海海事大学
邮箱	huangzhiwen@stu.shmtu.edu.cn
GitHub	https://github.com/Juwan-Hwang
申请日期	2026.08.03

一、项目背景

1.1 问题概述

Jeandle 是基于 LLVM 的 Java JIT 编译器。LLVM 完成代码生成后产出 ELF object, Jeandle 在 `JeandleCompiledCode::finalize()` 中解析该 object, 将机器码、常量、重定位信息及运行时生成的 stub 等内容安装到 HotSpot `CodeBuffer`, 最终进入 JVM Code Cache。

当前 `finalize()` 为 `CodeBuffer` 使用固定经验值进行预分配。其中 `consts` 区采用：

```
uint64_t consts_size = 6144 * wordSize;
```

在 RISC-V64 上约为 48 KiB。该策略存在两类问题：

- Code Cache 浪费：**大量小方法实际只使用少量 `.rodata` / `.data.rel.ro` 常量，但仍会预留约 48 KiB `consts` 空间。
- 扩容导致安装路径变慢：**当预分配不足时，`CodeBuffer::expand()` 会触发新的 `BufferBlob` 分配、内容拷贝和 relocation 修正；该操作在 `finalize` 过程中可能被多次触发，显著拖慢编译速度。

1.2 项目目标

设计并实现一个在 `JeandleCompiledCode::finalize()` 分配 Code Buffer 时的大小估算机制，将容纳 Jeandle 编译产出代码所需的 Code Buffer 大小精确求出，帮助 Code Buffer 一次性分配到位。具体目标：

- 精确度量 Jeandle 编译产物的各个段大小：遍历 ELF 段表，精确累加最终被保留到 Code Buffer 中的各个段的 `sh_size` 及其对齐 padding。
- 根据各段大小计算 Code Buffer 所需的容量大小。
- 一次性分配到位，消除常规路径的 `expand()` 触发。
- 通过现有测试保证功能正确性无回退；量化收益——统计 `expand` 触发次数（目标：常规路径降为 0）、内存浪费率、单方法 `finalize` 耗时，给出前后对比数据。

二、技术方案

2.1 问题分析：为什么不能简单累加 ELF section size

2.1.1 当前 consts 分配的问题

现有 consts 分配采用固定经验值，但 LLVM object 中每个方法的常量区差异很大：

方法类型	典型 const section 情况
小 getter / setter	几乎没有 const section
普通方法	少量 <code>.rodata</code>
复杂内联方法	多个 <code>.rodata</code> / <code>.data.rel.ro</code> section
含 oop、metadata、外部引用	存在不同 relocation 需求

因此，固定约 48 KiB 的 consts 预留会在小方法上显著浪费 Code Cache；而固定值又不能从语义上保证一定足够。

2.1.2 不能简单累加 section size

以下计算不正确：

```
consts_size = Σ section.sh_size
```

因为不同 const section 可能具有不同的 `sh_addralign`，在同一个 CodeBuffer consts section 中排布时需要插入 padding。

以下计算虽然是安全上界，但不是实际布局：

```
consts_size = Σ section.sh_size + Σ (section.sh_addralign - 1)
```

真正容量取决于：哪些 ELF section 会被装入 consts、各 section 的写入顺序、每个 section 的对齐要求、CodeBuffer consts section 的实际基地址和对齐保证、以及当前 const section 在 CodeBuffer 内的实际 offset。

2.1.3 关键陷阱：写入顺序不是 ELF section table 顺序

当前 const section 可能在 relocation 解析期间，由 `lookup_const_section()` 首次被访问时才加入内部 const section 集合。因此其实际写入顺序可能受以下因素影响：

- LinkGraph block 遍历顺序
- LinkGraph edge 遍历顺序
- relocation resolver 的处理顺序
- 某个 const section 首次被引用的时机

若 planner 按 ELF section table 顺序计算 padding，而 emitter/resolver 按首次引用顺序写入，则会出现：consts padding 预估错误、consts 预估不足仍触发 `expand()`、planned / actual 数据无法解释等问题。

因此，本项目 P0 的关键不是“扫描 section 并求和”，而是：

将 const section 的布局从隐式、由 relocation 首次访问决定的行为，改造成显式、确定、被多个路径共享的 layout plan。

2.2 总体设计

2.2.1 设计选择：确定性 canonical const layout

本项目采用**方案 (b)：确定性的 canonical const layout**，而非方案 (a)：每次 `finalize()` 都按 relocation 顺序额外 dry-run 一遍。

原因：

1. dry-run 会在 finalize 热路径上多一次遍历；
2. dry-run 与真正 resolver 若逻辑发生漂移，仍可能出现布局不一致；
3. 按首次访问顺序决定布局是隐式行为，维护成本高；
4. 预分配、复制和 relocation 应共享一个明确的事实来源。

但在开发和验证阶段，仍会保留轻量级的 traversal tracing，用于记录旧实现中 const section 的首次访问顺序、对比 canonical order 与旧顺序、验证改动前后 relocation target 正确性。

2.2.2 Canonical const layout 设计

P0 将构造一份 `ConstSectionPlan`：

```
struct ConstSectionPlanEntry {
    uint32_t elf_section_index;
    llvm::StringRef name;
    uint64_t size;
    uint64_t alignment;
    uint64_t consts_offset;
};

struct ConstSectionPlan {
    GrowableArray<ConstSectionPlanEntry> entries;
    uint64_t total_size;
    uint64_t max_alignment;
};
```

该 plan 是以下路径的**唯一事实来源**：

- CodeBuffer consts capacity calculation
- CodeBuffer consts section initialization
- `lookup_const_section()`
- `emit_consts()`
- relocation resolver
- debug verification
- statistics output

2.2.3 Canonical order 选择

```
筛选出实际需要装入 CodeBuffer consts 的 section
↓
按 ELF section index (section-header table 物理顺序) 排序
↓
按确定顺序计算每个 section 的 consts_offset
```

选择 ELF section index 的原因：顺序稳定；可在 allocation 前仅扫描 ELF 得到；不依赖 LinkGraph edge 的遍历顺序；便于在 planner 和 emitter 中共享；便于日志、测试和复现。

该选择必须在实现前验证：

1. Jeandle 当前需要复制到 consts 的 section 是否都可由 ELF section 唯一识别；
2. relocation resolver 是否能在 canonical layout 下正确查询各 section 的目标地址；
3. 更改 section 在 consts 中的相对排布后，oop、metadata、external const 和 section word relocation 是否正确；
4. `emit_consts()` 与 `set_const_section_alignment()` 的行为是否与该 layout 一致。

2.3 P0 实现方案：Consts 确定性布局

2.3.1 第一步：测量与基线

在修改 allocation 逻辑前，先建立可复现的 measurement infrastructure。每次 Jeandle 成功编译并进入 `finalize()` 后，记录：

```
method name, is_osr
initial insts/consts/stubs/locs capacity
actual insts/consts/stubs/locs usage
expand count by section, expand copied bytes by section
finalize elapsed time
const section list: ELF section index, name, sh_size, sh_addralign, actual consts offset
```

新增可选 JVM 参数：

```
-XX:+JeandlePrintCodeBufferStats
-XX:+JeandleTraceCodeBufferExpand
-XX:+JeandleDumpLayoutJson
```

该基线数据用于证明固定 consts 预留的实际浪费、找出 expand 发生在哪个 section、确定最常见的 const section 类型、确定 P0 前后对比样本。

2.3.2 Const section 分类

新增公共分类函数，放置在 `jeandleReadELF.hpp` 或现有 ELF 解析相关文件中：

```
static bool is_jeandle_const_section(llvm::StringRef name) {
    return name == ".rodata" ||
           name.starts_with(".rodata.") ||
           name == ".data.rel.ro" ||
           name.starts_with(".data.rel.ro.");
}
```

最终筛选条件必须与实际写入路径一致，不得只依据名称猜测。明确不直接进入 CodeBuffer consts payload 的 section: `.llvmstackmaps`、`.llvmfaultmaps`、`.gccexcepttable`、`.stacksizes`、`.rela.*`、`.symtab`、`.strtab`、`.shstrtab`、`.debug.*`、`.note.*`、`.comment` 等。

2.3.3 对齐与容量计算

ELF section 的 `sh_addralign` 要求 section 的最终装载地址满足对齐约束。实现前必须确认：

1. `CodeBuffer::initialize_consts_size()` 后 consts section base address 的对齐；
2. `set_const_section_alignment()` 是否影响 consts base address 或仅记录对齐需求；
3. `CodeEntryAlignment` 与所有相关 `sh_addralign` 的关系。

若 consts base 对所有 section alignment 都已满足，则可按相对 cursor 排布：

$$offset_{i+1} = \text{align_up}(offset_i, alignment_i) + size_i$$

否则必须基于绝对地址求 offset。

计算函数伪代码：

```
bool compute_const_section_layout(
    ConstSectionPlan* plan,
    uint64_t consts_base_alignment);

uint64_t cursor = 0;
for (ConstSectionPlanEntry& entry : plan->entries) {
    // 校验: alignment 非零、为二次幂、不超过 CodeEntryAlignment
    cursor = align_up(cursor, entry.alignment);
    entry.consts_offset = cursor;
    // 溢出检查
    cursor += entry.size;
}
plan->total_size = cursor;
```

必须检查：alignment 非零、为二次幂、不超过 CodeBuffer 已支持的边界、对齐计算不溢出、容量累计不溢出、每个 section 的 offset 不重叠、最终 size 与实际 copy 后的 cursor 一致。

2.3.4 CodeBuffer 初始化协议

当前 Jeandle `CodeBuffer::initialize()` 使用四参数形式，随后调用 `initialize_consts_size()`。P0 中使用 `InstallLayoutCore` 结构体封装各分区容量：

```
struct InstallLayoutCore {
    size_t insts_payload;
    size_t consts_payload;
    size_t stubs_payload;
    size_t locs_payload;
    size_t code_size_input;
    bool used_legacy_fallback;
};
```

P0 中仅替换 consts 固定 48 KiB 估算：

```
consts_payload = ConstSectionPlan.total_size
insts_payload  = 保持当前安全估算
stubs_payload  = 保持当前安全估算
locs_payload   = 保持当前安全估算
```

2.3.5 修改 lookup_const_section 与 emitter

改造后 `lookup_const_section()` 不再决定 section 的插入顺序，仅根据 section identity 查询

`ConstSectionPlan`，返回已确定的 `consts_offset` 与目标地址。优先使用 ELF section index 而非 section name，因为 section index 与 ELF object 的身份关系更稳定。

实际 const copy 与 relocation 修正必须共同使用 `entry->consts_offset`，不得在某一处重新累加 section size 或重新推导 padding。

2.3.6 Relocation 正确性验证

改变 const section 排列顺序后，必须验证所有 relocation 在修正后仍指向 canonical const layout 中目标 section 的正确位置。在 fastdebug / develop build 中增加：

```
assert(target >= consts_begin, "target must be inside consts section");
assert(target < consts_end, "target must be inside consts section");
assert(target == consts_begin + plan_entry.consts_offset + local_offset,
        "relocation target must match planned const section layout");
```

重点验证的 relocation 场景包括：`SectionWordReloc`、`OopReloc`、`OopAddrReloc`、PC-relative relocation、external const reference、metadata reference、static call related reference、RISC-V HI20 / LO12 paired relocation。

若某种 relocation 引用了一个未被 `ConstSectionPlan` 收录的 section，fastdebug 下输出诊断并 ASSERT，product 下放弃本次精确 planner，使用 legacy fallback。

2.4 P1：其他分区容量收紧

在 P0 稳定后，逐步改进：

2.4.1 Insts 容量规划

将 instruction section 的容量拆解为：

```
$$insts_payload = elf_text_size + prolog_upper_bound + entry_alignment_padding +
runtime_handler_upper_bound$$
```

RISC-V64 prolog 可能包括 OSR entry、IC check、verified entry、clinit barrier、stack overflow check、nmethod entry barrier、entry alignment padding。采用"条件分类 + 每组件最大长度 + 最大可能对齐 gap"策略，每项有对应的真实 emitter 代码位置和 fastdebug 下 `actual <= planned` 的校验。

2.4.2 Stubs 容量规划

stubs section 可能包含 static/dynamic/external call stub、trampoline、far branch stub、entry barrier stub、exception handler、deoptimization handler 等。P1 先建立 stub 统计，对已知静态 stub 按上界一次预留，保留动态 trampoline 的可解释 expand 路径。不将 `stubs expand = 0` 作为硬约束。

2.4.3 Locs 容量规划

LLVM LinkGraph edge 数量不等于最终 HotSpot `relocInfo` 的数量或字节数（RISC-V 存在配对重定位、一对多映射、变长 encoding 等）。P1 引入逐类统计 `RelocPlan`，每类 relocation 映射到 HotSpot `relocInfo` 类型、实际编码大小和附加 payload。在未完成该模型前保持现有保守估算。

2.5 安全机制

2.5.1 Legacy fallback

以下情况必须放弃 canonical const layout，并使用当前保守分配策略：

- ELF section table 解析失败
- 必要 const section 无法识别
- section alignment 非法
- section size 或总容量发生 overflow
- `ConstSectionPlan` 与实际 resolver 发现的 section 集不一致
- 未知 relocation 访问了未规划 const section
- CodeBuffer 初始化失败

每次 fallback 必须可统计（fallback_count、fallback_reason、method、section / relocation kind）。

2.5.2 Expand 保留策略

本项目不移除 `CodeBuffer::expand()`。其定位从"预期的常规扩容机制"变为"生产环境最后一道安全网"。P0 成功标准是 `consts section expand = 0`，P1 再逐步收紧 insts、stubs、locs。

2.6 测试方案

2.6.1 单元测试

覆盖：单一/多个 `.rodata` section、`.data.rel.ro` section、section index 顺序与旧首次引用顺序不同、各种 alignment（含非法）、容量 overflow、未知 section、`ConstSectionPlan` 查询、canonical layout offset 计算、planner/emitter offset 一致性、fallback 行为。

2.6.2 jtreg 测试

在 `test/hotspot/jtreg/compiler/jeandle/` 下新增：

测试	验证重点
TestCodeBufferConstLayout	canonical const order、alignment、planned/actual offset
TestCodeBufferConstsNoExpand	成功 planner 下 consts 不扩容
TestCodeBufferFallback	planner 失败时 JIT 仍正确执行
TestCodeBufferInstsNoExpand	P1 后 insts 不扩容
TestCodeBufferStubsStats	stubs/trampoline 统计可用
TestCodeBufferOSR	OSR prolog reservation
TestCodeBufferBarriers	clinit / nmethod entry barrier
TestCodeBufferCalls	static/dynamic/external call
TestCodeBufferExceptionDeopt	exception、deopt、fault map
TestCodeBufferStress	高频编译、小 Code Cache、CodeBuffer pressure

2.6.3 压力测试

使用 `-Xcomp`、`-XX:+StressCodeBuffers`、小 Code Cache、fastdebug、长时间运行、高频编译、高常量密度、深度内联、异常路径、OSR、MethodHandle。重点检查 CodeCache corruption、relocation target 错误、SIGSEGV/SIGILL、assert failure。

2.7 Benchmark 方案

指标

类别	指标
内存	allocated、actual used、slack、waste rate
扩容	expand count、expand bytes、copied bytes、按 section 分类
时延	finalize 平均值、P50、P95、P99
编译	每千次 JIT compile 的 expand 次数
正确性	jtreg、fastdebug、压力测试结果
fallback	fallback 次数、原因、涉及方法比例

Benchmark 集

- Microbenchmark：getter/setter、常量密集方法、多 rodata section 方法、data.rel.ro 方法、深度内联、调用密集、异常路径、数组访问、OSR loop、MethodHandle
- Java workload：DaCapo、Renaissance、SPECjvm2008（环境允许时）

环境说明

优先级：RISC-V64 真机若可作为最终性能验证优先环境；RISC-V64 QEMU 作为功能、jtrg、fastdebug 验证。QEMU 结果用于比较优化前后的相对趋势，不直接解释为真实 RISC-V 硬件上的绝对吞吐或延迟结论。

三、开发时间计划

本项目以 12 周（约 3 个月）为周期，按周划分里程碑。每周 40+ 小时投入，预留约 15% 弹性时间应对 Debug。开发周期划分为 P0 阶段（Week 1-6）和 P1 阶段（Week 7-12）。

P0 阶段：Consts 确定性布局 (Week 1-6)

Week 1: 环境、闭环与 baseline

类别	内容
Coding	搭建 RISC-V64 QEMU 环境; 构建 Jeandle JDK fastdebug 与 product 版本; 定位 finalize()、CodeBuffer::initialize()、initialize_consts_size()、expand()、lookup_const_section()、emit_consts()、relocation resolver
Testing	运行最小 Java 程序验证 JIT 安装链路; 跑通 microbenchmark; 跑通现有 compiler/jeandle jtrg
Docs	增加初始 instrumentation; 收集 legacy fixed-size 策略 baseline JSON 数据; 绘制 CodeBuffer 安装路径图

交付物：RISC-V64 环境文档、Hello/microbenchmark 闭环结果、baseline JSON 数据、CodeBuffer 安装路径图

Week 2: Const section 现状分析与 alignment 调查

类别	内容
Coding	梳理当前 lookup_const_section() 的 section 发现和插入顺序; 提取 is_jeandle_const_section()
Testing	确认真实进入 consts 的 ELF section 类型; 验证 ELF sh_addralign 与 CodeBuffer alignment 的关系
Docs	记录实际 relocation 驱动的首次引用顺序; 确认 set_const_section_alignment() 和 initialize_consts_size() 对 base address 的影响

关键验收问题：consts base address 是否满足每个 const ELF section 的 sh_addralign? section placement 应按 relative offset 还是 absolute address 对齐? canonical order 是否可安全采用 ELF section index?

交付物：const section 分类说明、首次引用顺序 trace、alignment 调查报告、section classifier 单元测试

Week 3: ConstSectionPlan 与 canonical layout

类别	内容
Coding	实现 ConstSectionPlan；根据 ELF section index 建立稳定顺序；计算 consts_offset、total_size、max_alignment；加入 alignment、overflow 和 section identity 校验
Testing	layout calculation 单元测试
Docs	输出 canonical layout 与当前实际写入顺序的对照日志

交付物：ConstSectionPlan 初版、layout calculation 单元测试、planned/current-order 对照日志

Week 4: Planner 与 emitter 共享布局

类别	内容
Coding	修改 lookup_const_section()，将"首次访问时 append section"的隐式行为替换为 plan lookup；让实际 const copy 使用 plan 的 offset；让 relocation resolver 使用同一 plan
Testing	加入 relocation target 范围与 offset 一致性 ASSERT；验证 section 重排后 Java 语义正确
Docs	TestCodeBufferConstLayout 测试

交付物：共享 const layout 的 resolver/emitter 改造、fastdebug relocation assertions、TestCodeBufferConstLayout

Week 5: P0 接入 CodeBuffer 与 fallback

类别	内容
Coding	使用 ConstSectionPlan.total_size 替换固定 consts 预留；正确计算 code_size_input；实现 legacy fallback；记录 fallback 原因
Testing	加入 consts planned/actual 统计；TestCodeBufferConstsNoExpand；首轮 RISC-V64 jtreg
Docs	P0 前后初步对比数据

交付物：P0 可运行实现、fallback 实现、consts no-expand 测试、P0 前后初步对比数据

Week 6: P0 压测与首个 PR

类别	内容
Coding	使用 -Xcomp、-XX:+StressCodeBuffers、小 Code Cache、fastdebug 覆盖高常量密度、多 const section、多 relocation case；修复发现的问题
Testing	压力测试与修复；整理并提交第一个小型、独立可 review 的 PR
Docs	PR 范围：instrumentation 基础设施、ConstSectionPlan、section classifier、canonical const layout、legacy fallback、P0 jtreg 测试

交付物：P0 PR、fastdebug/jtreg/stress 测试结果、P0 benchmark 初版

P1 阶段：其他分区容量收紧（Week 7-12）

Week 7：RISC-V64 prolog 调查与 insts 模型

类别	内容
Coding	枚举 RISC-V64 prolog 发射分支（OSR entry、IC check、verified entry、clinit barrier、stack overflow check、nmethod entry barrier、entry alignment）；实现 <code>prolog_upper_bound()</code>
Testing	测量典型 Java 方法的 prolog 实际大小；增加 prolog planned/actual 统计；OSR/barrier jtreg 测试
Docs	RISC-V64 prolog 字节级分析

交付物：RISC-V64 prolog 分析、insts reservation 初版、OSR/barrier jtreg 测试

Week 8：Handler 与 stubs 模型

类别	内容
Coding	分析 exception、deopt、implicit exception、MethodHandle 路径；分析 static/dynamic/external call stub、trampoline/far branch；建立 stubs 分类统计；实现已知静态 stub 的上界预留
Testing	<code>TestCodeBufferStubsStats</code>
Docs	stub 分类表

交付物：stub 分类表、handler/stubs reservation 初版、`TestCodeBufferStubsStats`

Week 9：Relocation / locs 调查与分类

类别	内容
Coding	复用当前 resolver 所用的 LinkGraph；统计 RISC-V relocation edge；映射 Jeandle relocation 类型与 HotSpot relocInfo 编码；区分 PCREL HI20/LO12 pair、direct call、external call、oop、metadata 等
Testing	输出 reloc distribution；对未知类别使用 legacy locs fallback
Docs	relocation 分类文档；locs planned/actual 统计

交付物：`RelocPlan` 原型、relocation 分类文档、locs planned/actual 统计

Week 10：P1 集成与分区验证

类别	内容
Coding	接入 insts reservation；接入静态 stubs reservation；接入已验证 relocation 类型的 locs reservation；分区统计 expand

类别	内容
Testing	验证 consts expand = 0、insts expand = 0、stubs/locs expand 可解释可统计；修复 underestimation； <code>TestCodeBufferInstsNoExpand</code>
Docs	分区 expand 报告

交付物：P1 集成实现、分区 expand 报告、 `TestCodeBufferInstsNoExpand`

Week 11: 完整 benchmark 与长稳测试

类别	内容
Coding	运行 microbenchmark、DaCapo、Renaissance、可选 SPECjvm2008；运行 fastdebug 长时间压力测试
Testing	汇总 waste rate、expand rate、copied bytes、finalize P50/P95/P99、fallback rate
Docs	原始 benchmark 数据、复现脚本、对比表格与图表、稳定性报告

交付物：原始 benchmark 数据、复现脚本、对比表格与图表、稳定性报告

Week 12: PR 拆分、文档与最终交付

类别	内容
Coding	整理 PR 序列，确保每个 PR < 300~500 行有效逻辑改动、单一主题、可独立构建、附对应测试
Testing	最终全量 jtreg + fastdebug + stress 验证
Docs	P0/P1 设计文档、benchmark 脚本、结果文档、性能对比报告

建议 PR 序列：

- 1. Instrumentation PR — CodeBuffer usage/expand/finalize timing telemetry
- 2. Const layout PR — section classifier、ConstSectionPlan、canonical layout、resolver/emitter shared mapping
- 3. P0 allocation PR — exact consts allocation、legacy fallback、ASSERT、jtreg
- 4. RISC-V insts PR — prolog/handler reservation、insts no-expand tests
- 5. Stubs and locs PR — stub classification、relocation classification、capacity refinement
- 6. Benchmark and documentation PR — benchmark scripts、result documentation、design document

最终交付物：完整源代码、P0/P1 设计文档、jtreg 测试、fastdebug/stress 测试报告、benchmark 脚本、原始测量数据、性能对比报告、上游 PR 链接

四、风险评估与应对

风险	影响	应对
Canonical order 漏掉某个实际使用的 const section	relocation target 错误或 consts 不足	Section identity 校验、fastdebug ASSERT、product fallback
以 section name 识别 section 存在歧义	查询错误	优先使用 ELF section index
ELF sh_addralign 与 CodeBuffer base alignment 不兼容	未对齐访问或 padding 算错	Week 2 优先验证绝对地址对齐语义
改变 const section 相对位置破坏 relocation	JVM 错误执行	shared plan、relocation target ASSERT、jreg、stress test
planner 与 emitter 逻辑漂移	planned/actual 不一致	两者仅查询同一份 ConstSectionPlan，不分别计算 offset
新增未知 relocation 类型	locs 低估	unknown type 触发 legacy locs fallback
RISC-V prolog emitter 变化	insts 上界失效	上界代码贴近 emitter；actual <= planned ASSERT
QEMU 性能数据失真	benchmark 结论不可靠	用 QEMU 验证相对趋势；真机优先用于最终性能数据
代码范围膨胀	3 个月无法完成	P0 独立交付优先；P1 可拆分、可降级

五、个人准备与能力说明

5.1 已完成的准备工作

- 环境搭建：**已在 x86_64 主机上，使用 riscv64-linux-gnu-gcc 交叉编译工具链和 riscv64 sysroot，完整走通了 jeandle-jdk 面向 RISC-V64 的交叉编译。此前没有从源码构建过 OpenJDK 或同等规模 C++ 项目，此次是第一次完整走通。
- 代码调研：**已梳理 JeandleCompiledCode::finalize() 的现有代码逻辑，定位了 CodeBuffer::initialize()、initialize_consts_size()、expand()、lookup_const_section()、emit_consts() 和 relocation resolver 等关键路径。
- 导师沟通：**已与导师何学锦老师进行多轮邮件交流，确认了以下关键设计决策：
 - 采用方案 (b) 确定性 canonical const layout，避免 dry-run 在 finalize 热路径上增加额外遍历；
 - padding 对齐来源需优先验证 CodeBuffer consts section base address 的对齐保证；
 - 支持 P0/P1 渐进式实现和 PR 拆分；
 - 将"先建立可复现的测量手段"作为真正的第一步。

5.2 工程能力

- **大型项目独立维护经验：**独立开发并维护 Zephyr (GitHub 666 stars)，基于 Tauri + Rust 的跨平台桌面客户端，涉及完整的 CI/CD 流程搭建、跨平台构建与发布、测试体系维护和安全审计。这部分积累了在工程管理和独立维护复杂项目上的主要经验。
- **系统级编程转型：**此前的开源和工程经验主要集中在 Rust 应用开发方向，C++ 和系统级编程是本次主动补齐的方向。Rust 与 C++ 在底层内存模型、指针语义、unsafe 边界等方面有相通之处，可以较快迁移。
- **时间保障：**OSPP 周期内每周可稳定投入 40+ 小时，无实习、课程、考试或毕设等其他安排。